

Aayush Sangani

U.S. Citizen | (916)-294-5038 | sanganiaayush@gmail.com | [sanganiaa](https://github.com/sanganiaa) | in/aayush-sangani | aayushsangani.me

EDUCATION

University of California, San Diego

La Jolla, CA

Bachelors of Science in Computer Engineering

Expected June 2027

- **Relevant Coursework:** Computer Architecture, Digital Logic Design, ASIC and FPGA Design, Microarchitecture, Advanced VLSI, Systems Programming, Data Structures, Linear Algebra

TECHNICAL SKILLS

Hardware & HDL: SystemVerilog, Verilog, ARM/RISC-V Assembly, x86 Assembly, RTL Design, Physical Design

EDA/CAD Tools: Synopsys Fusion Compiler, PrimeTime, Cadence Innovus, Tempus, QuestaSim, Quartus

Languages: C/C++, Python, TCL, Perl, Bash/Shell, TypeScript, JavaScript, SQL, Java, HTML/CSS

Frameworks & Tools: PyTorch, scikit-learn, pandas, React, FastAPI, Docker, AWS, PostgreSQL, Linux, Git

EXPERIENCE

Intel

Folsom, CA

Hardware Engineering Intern – CPU Physical Design

June 2026 – Present

- Running **multi-corner STA** in **PrimeTime**, identifying critical path violations and driving **ECO** to close **slack**
- Executing **cluster-level PnR** on **P-core SMT-enabled CPU** blocks using **Synopsys Fusion Compiler**
- Analyzing setup/hold timing at **medium PVT corners**, constraining **hard IP** boundaries and **I/O interfaces**
- Performing **floorplanning** and **CTS** to balance clock skew and converge **RTL-to-GDS** flow on **CPU** clusters
- Automating full-chip/block-level STA via an **MCP AI** agent with custom **Tcl/Python** scripts and **Markdown** skill definitions, identifying **setup/hold** fixes on **critical paths**, cutting analysis time from **10-15 min** to **5 min**

California Franchise Tax Board

Sacramento, CA

Software Engineering Intern

January 2025 – September 2025

- Automated disk audits via **PowerShell**, executing **SQL Server** 2016–2022 migrations across **10+** environments
- Developed **PowerShell** and **SQL** scripts to migrate logins and identify linked server dependencies
- Configured service accounts, permissions, and firewall rules with Security and File-app teams for **SQL Server** 2022
- Maintained parallel fallback servers for **1–2** months, completing final cutover after validating all dependencies

Key Business Solutions

Sacramento, CA

Software Developer Intern

July 2024 – August 2025

- Diagnosed and resolved **React** state mutation bugs across **15+** chatbots, eliminating recurring UI defects
- Audited **API** call chains via **Chrome DevTools** and triaged client-reported bugs across **15+** chatbots via **Jira**
- Deployed frontend builds on **AWS EC2**, isolating failures via **PM2** logs; reducing resolution under **30 minutes**

PROJECTS

Argus – Parameterized N-bit LFSR BIST Architecture 🔗

- Built a self-testing hardware block to reduce the need for expensive external **ATE**, embedding on-chip pattern generation and output compaction to reduce silicon test time and improve **fault coverage** at **tapeout**
- Designed a parameterized N-bit **LFSR** in synthesizable **SystemVerilog RTL** as a **PRPG**, generating $2^N - 1$ deterministic test patterns through configurable **tap logic**, **seed initialization**, and **XOR feedback**
- Extended to a full **BIST** architecture with **MISR** output compactor and **FSM** test controller, validating **stuck-at fault** detection through injection in **QuestaSim** with **SVA** assertions and **waveform analysis**
- Synthesized in **Quartus** across $N=4$ and $N=8$, analyzing **LUT/FF** utilization, **critical path** timing, and **Fmax** to evaluate **PPA** tradeoffs across **FPGA** and **ASIC**-oriented physical design parameter sweeps

Midas – Financial News Intelligence Platform 🔗

- Built a **FastAPI** pipeline scoring news and **SEC EDGAR** filings into **BUY/HOLD/SELL** signals per ticker
- Deployed **FinBERT** via **PyTorch**, persisting **time-weighted** scores to **PostgreSQL** on an **APScheduler** job
- Ingested **Yahoo/Google RSS** and **Finnhub** news feeds, deduplicating and cleaning articles before inference
- Containerized with **Docker**, exposing a **REST API** and **Python CLI** over the ingestion pipeline